Knowledge Outside the Prompt

Processes built on large language models: the fourth session of the arc.

What Carries Over from Last Session

The process you left last session with could do five things: answer the requester instead of staying silent; check its own answer before sending it; decide the refund amount by a rule written in code instead of letting the model guess; answer in one stable format on every call; and, if a photo is attached to the request, read it instead of just saving it as an attachment.

The question S9 ended on, verbatim: "At what size and volatility does the knowledge base make 'everything in the prompt' stop being reasonable?"

Olga's training center doesn't stand still: something is added to the base every week — a new regulation, an updated FAQ, a breakdown of another disputed case. S9's ten items are the base's state at the moment the process launched; from there it grows.

The Session's Axis: Where the Process Gets Its Facts and Why It Considers Them True

LOOKS RELIABLE	WHAT IT TURNS OUT TO BE
if the base has grown — just list it all in the prompt, like on S9	
you can cut a document into pieces any way you like, as long as the chunk ends up in storage	
semantic search (vector) finds the same thing as keyword search anyway, just smarter	
if we know how to search text, we'll find an image the same way, just by captioning it in words	

Where does the process get its facts, and why does it consider them true?

Four rows — the session's four cases. The right column is filled in with measurement: a run of our own rig on real data.

Four Cases and One Interlude

1. **Knowledge Base Storage** — what happens to the answer's accuracy and cost as the base grows from ten items to hundreds of documents.
2. **Splitting a Document into Chunks** — once the knowledge base turns from ten lines into real documents, they first need to be cut up; whether the answer to a question stays intact depends on where the cut falls.
3. **Choosing a Search Method** — once there are many chunks in the base, the search method (by words or by meaning) isn't interchangeable.
4. **Image Search** — the same storage-and-search task, only for the scans and photos the training center receives in requests.

Interlude: a system can find a topically relevant chunk and still give a wrong answer. "Found" and "correct" are different things.

Case 1. Knowledge Base Storage

The training center's knowledge base grows every week: a new regulation, an updated FAQ item, another disputed case worked through. Seminar 9's ten items were the starting point; from here the knowledge base only grows.

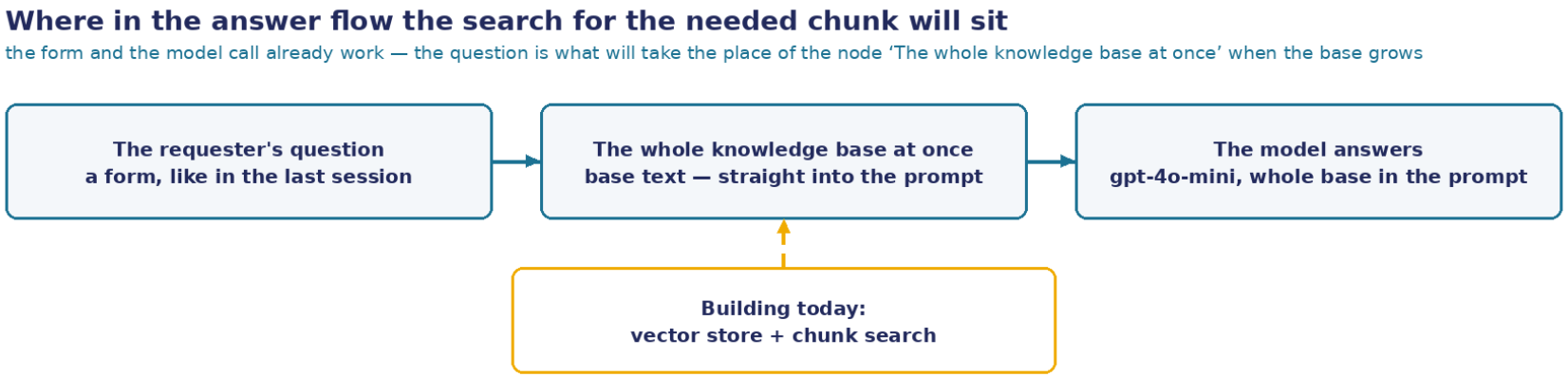
Хранение базы знаний

Разбиение документа на части

Чем искать нужный кусок

Поиск по изображениям

What Olga needs from a growing base



The need. The answer stays accurate and cheap even once the base grows from ten items to hundreds of documents.

1. Version 1 — the natural continuation of Seminar 9: keep appending the base to the same prompt, with no ceiling.
2. Two more real methods: search by word match; search by meaning — through a vector store.

All three are real engineering options. Which one survives the base's growth is the next screen's question.

How to build a knowledge base that grows

How to build a knowledge base that grows?

keep appending the base to the same prompt

search by keyword

put it in a vector store and pull out the needed chunk

What each of the three actions does

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Keep appending the base to the same prompt	The whole text of the base goes to the model, in full, on every question again	Needs no preparation at all in advance — the base just sits as text in the process's settings	The bigger the base, the more text goes out on every question — the cost grows together with the base
Search by keyword	The overlap of the query's words and the document's text, with not a single call to the model	Cheap and fast regardless of base size; the preparation — indexing by word — is done once, before the first question arrives	Finds a word match: meaning plays no role here — a question phrased in different words may miss the chunk where the answer actually lives
Put it in a vector store and pull out the needed chunk	The base is turned in advance into numeric representations of meaning; a query finds whatever is closest in meaning, with no exact word match needed	Only the found chunk goes into the prompt — the size of the whole base doesn't affect the cost	Needs a separate preparation step in advance: the base has to be split into chunks and an embedding computed for each, before the first question arrives

Measurement structure: ten questions, four sizes, three actions

Model and settings. `gpt-4o-mini` — answer from the passed-in text; `text-embedding-3-small` — embedding for the search. Both with no explicitly set additional settings, the same call across all twelve combinations.

WHAT VARIES	VALUES
Base size	10 · 50 · 200 · 500 documents
Action	the whole prompt · search by word → answer · search by meaning → answer
Questions	the same set of 10, identical across every size and action

Example. "What is the name of the circulation circuit that runs through the lungs?" — correct answer: "the pulmonary" (document `needle-1`, present in the base at all four sizes).

What the run showed at four base sizes

ACCURACY	WHOLE PROMPT	KEYWORD → ANSWER	VECTOR → ANSWER
10 docs	0.7	0.5	0.6
50 docs	0.7	0.5	0.7
200 docs	0.7	0.4	0.6
500 docs	**HTTP 400 ERROR**	0.5	0.7

$/QUESTION	WHOLE PROMPT	KEYWORD → ANSWER	VECTOR → ANSWER
10 docs	$0.000189	$0.0000484	$0.0000486
50 docs	$0.000809	$0.0000484	$0.0000482
200 docs	$0.003204	$0.0000501	$0.0000481
500 docs	— rejected	$0.00005	$0.000049

Retrieval recall@1: keyword — **0.8 across all four sizes without change**; vector — **1.0 at 10/50/200, 0.9 at 500**. **At 500 documents, prompt-stuffing needs around 160,000 tokens — more than gpt-4o-mini's context (128,000); OpenRouter rejects the request with HTTP 400 before the model ever answers.**

Keyword's systematic error: mixes up case endings

Error 1 — keyword, case endings (identical at every base size)

QUESTION	OVERLAP WITH THE CORRECT DOCUMENT	OVERLAP WITH THE FOUND (WRONG) DOCUMENT	VERDICT
q4: "...by paleontologists Niles Eldredge and Stephen Jay Gould..." → gold answer "1972"	needle-4: "...in 1972 paleontologists Eldredge and Gould..." — 1 word ("year")	needle-2 (about Mozart): 1 word ("year") — a tie, the wrong one was picked	**INCORRECT**
q10: "...whose model of the universe was taken as a basis by Kepler?" → gold answer "Copernicus"	needle-10: "...the Keplerian model of the universe is based on Copernicus's model..." — 1 word ("universe")	needle-3 (about a marathon): 2 words ("model," "universe") — the margin went against the correct one	**INCORRECT**

Keyword doesn't count case-form matches as a match — the error holds even at 10 documents with no distractors: the cause isn't the base's scale, it's the method.

Vector's systematic error: confuses neighboring paragraphs

Error 2 — vector, a neighboring paragraph of the same article (only at 500 documents)

QUESTION	FOUND DOCUMENT	MODEL'S ANSWER	VERDICT
q3: "What summer temperature keeps marathon runners from showing their best results?" → gold answer "high," document needle-3	`distractor-237` instead of needle-3 — likely a neighboring paragraph of the same article (shared footnote [17]) with a literal number	"Temperatures above 14-16°C make marathon runners' results worse."	**INCORRECT** (substantively sensible, against the gold answer)

Here keyword found the CORRECT needle-3 — but the model still answered "no information": there was no number in it; the number sat in the paragraph that went to the vector option instead.

What the measurement confirmed: appending everything to the prompt

The model that forms the final answer in all three actions is the same one: `gpt-4o-mini`, with no explicitly set call settings. Action 3 additionally uses `text-embedding-3-small` for the search.

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Append everything to the prompt	The whole text of the base goes to the model, in full, on every question again	Accuracy holds at one level, 0.7, at 10/50/200 documents — with no visible warning signal	Cost grows 17.0-fold from 10→200 documents at the same accuracy; at 500 documents it doesn't work at all — the request exceeds the model's context, with no prior quality drop

What the measurement confirmed: searching by word and by meaning

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Search by keyword	Overlap of the query's words and the document's, with no model	Cheap and fast regardless of base size (~$0.00005)	Retrieval recall@1 = 0.8 at all four sizes without change: holds exactly in place regardless of the base's volume — the cause is comparing words without accounting for case endings (breakdown — a separate beat)
Put it in a vector store	The query's embedding is compared against the base's chunk embeddings; only the found chunk goes into the prompt	The prompt holds ~270 tokens and ~$0.00005 regardless of base size; retrieval recall@1 = 1.0 at 10/50/200 documents	Needs a separate preparation step (split the base, compute embeddings) before the first question arrives; recall@1 dips to 0.9 at 500 documents — at that scale, neighboring paragraphs of the same article start confusing the semantic search (breakdown — a separate beat)

Where the base is stored and what the model sees

Instead of handing the model the whole base on every question, the process stores the base separately from the conversation. On every new query, it first finds several matching chunks in that separate store and puts only those into the prompt, leaving the rest of the base aside.

The real canvas: a vector store in n8n



Both the embedding step and the final model call are built with a plain HTTP Request node carrying a pre-set credential type — there's no stock "embedding via OpenRouter" node in n8n's catalog at all.

The real panel: the search node's code

JavaScript

```
const payload = JSON.parse($('Вопрос + корпус с векторами').first().json.payload);
const qvec = $input.first().json.data[0].embedding;
function cos(a,b){let d=0,na=0,nb=0;for(let i=0;i<a.length;i++){d+=a[i]*b[i];na+=a[i]*a[i];nb+=b[i]*b[i];}return d/(Math.sqrt(na)*Math.sqrt(nb));}
let best=null,bestScore=-1;
for(const doc of payload.corpus){const s=cos(qvec,doc.embedding);if(s>bestScore){bestScore=s;best=doc;}}
return [{json:{question:payload.question,chunk:best.text,doc_id:best.id,score:bestScore,model:payload.model}}];
```

The "Simple Vector Store" node is a loop over every document in the corpus and a hand-written cosine similarity function: no call to a separate service.

The real panel: the answer node's prompt

Prompt (User Message)

```
Отвечай ТОЛЬКО по тексту ниже. Если ответа нет в тексте — напиши «нет информации».

Текст:
{{ $json.chunk }}
```

> *"Answer ONLY from the text below. If the answer isn't in the text — write "no information." Text: `{{ $json.chunk }}`"*

What Olga sees, and what the requester sees

To the requester this is invisible entirely — they get an answer that's the same quality or better. Olga sees something different: at the current base size (while it's still small) the difference is barely noticeable, but it's exactly this choice that decides what each next answer will cost once the base grows.

The principle carries over to any orchestrator: n8n here is just an example. A prompt that holds the entire knowledge base grows in cost linearly with the base's size and doesn't guarantee rising accuracy — an external store, from which only the needed chunk gets pulled on every question, keeps the cost and the prompt's volume constant no matter how many documents have piled up.

When a store pays off, and when it doesn't

Criterion. An external store pays off when the base's growth makes prompt-stuffing either noticeably more expensive or structurally impossible — not through the mere fact that a base exists.

WHEN IT'S NEEDED

The base grows noticeably faster than once a year, and the count runs into hundreds of documents

The call's cost is already measurably above zero and shows up on a monthly bill

The base's size could one day exceed the model's context

WHEN IT ISN'T

The base is the same ten-to-twenty FAQ items as last session, and isn't planning to grow

The cost difference ($0.0002 vs. $0.00005) still doesn't justify a separate preparation step

The context limit is far off, with no structural failure in sight

A stale index — this rung's typical problem

A stale index is a typical problem of this rung, and this finding isn't specific to this one run. A vector store is a copy of the base as of the moment it was put there: it holds no live link to the source. If the regulation in the source document has changed and reindexing hasn't run yet, the store honestly finds and hands back a chunk — just a stale one.

The session axis's first line is closed: a modern model doesn't fail structurally as the base grows — it keeps getting steadily more expensive up to a certain volume, and then stops answering at all; an external store keeps cost and recall@1 = 1.0 constant at any of the sizes tested.

Case 2. Splitting a Document into Chunks

Case 1 decided where the base would live once it grew — in a separate store, outside the prompt. But the store doesn't swallow a document whole: a three-page regulation doesn't fit into one chunk in a way that lets search later point precisely at the right spot.

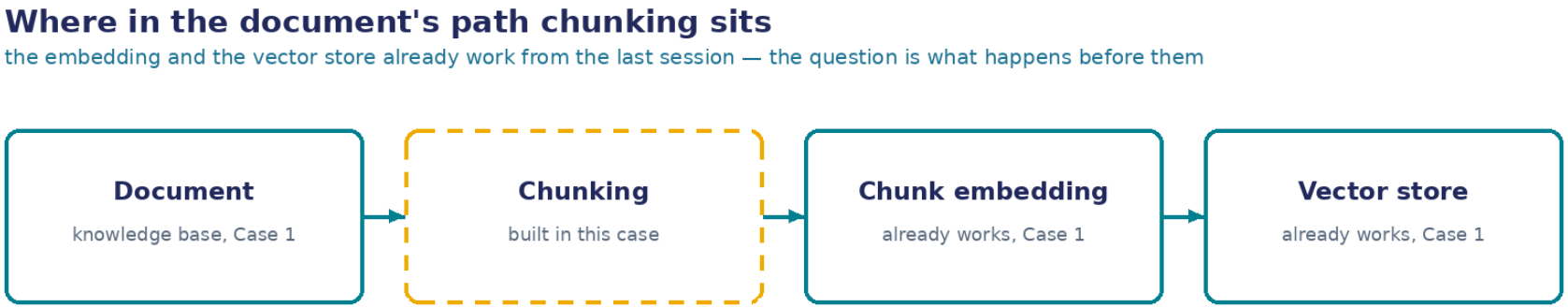
Хранение базы знаний

Разбиение документа на части

Чем искать нужный кусок

Поиск по изображениям

How to cut a document so a chunk doesn't lose meaning



The need. Every chunk in the store carries one complete piece of meaning — the answer to a question must not end up torn between two neighboring chunks.

Version 1 — the natural first attempt: cut the document into fixed-size chunks, by character count, regardless of sentence boundaries.

One parameter, one pass through the text. Whether this works for meaning — we check next.

Where should the document cut fall

Where should the document cut fall?

- **cut strictly by character count, without regard for the text**
- **cut by sentence boundaries**
- **cut by word boundaries**
- **cut by character count with overlap**

Two fundamentally different ways: by characters and by sentences

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Cut strictly by character count, without regard for the text	The document is chopped into pieces of a set length with no regard for the text inside	Simplest to implement — one parameter, one mechanical pass through the text	The cut can land exactly in the middle of the phrase holding the answer, regardless of the chosen chunk size; there's no way to tell in advance which size is "more convenient" without checking
Cut by sentence boundaries	The cut falls only where one sentence ends and the next begins	Chunks don't tear a phrase in half: the cut follows a boundary of meaning in the text — character count plays no role here	A bit more involved to implement: it needs a separate step that understands where a sentence ends in the given language — plain character counting isn't enough here; the quality of this step depends on the text

Two patches on the first method: by words and with overlap

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Cut by word boundaries	The document is split into words; a chunk accumulates until it exceeds the target size, and the cut falls only at the boundary between two neighboring words	Doesn't tear a word in half; simpler to implement than parsing sentence boundaries — all it needs to know is where a word ends	This method knows nothing about sentence boundaries: the cut can still land right in the middle of a phrase, even if no word is touched
Cut by character count with overlap	The same logic as cutting by character count, but neighboring chunks overlap — part of the characters at the end of one chunk repeats at the start of the next	Part of the answers torn at the first action's boundary end up falling entirely inside the neighboring overlapping chunk	The store grows due to character duplication; overlap doesn't guarantee a result for an answer of any length

Test structure: what was checked, and on what

Corpus. SberQuAD, 60 documents (no shorter than 500 characters — otherwise there's nothing to cut), 72 questions with answer labels verified directly against the text.

Finding along the way. In 22.0% of rows in the source dataset (1,108 of 5,036), the answer's position didn't match the document's text — such rows were excluded from the measurement entirely, with no adjustment for a convenient result.

Five groups — the same chunk size for all four versions: 120 · 180 · 220 · 300 · 450 characters.

180 is this session's headline value.

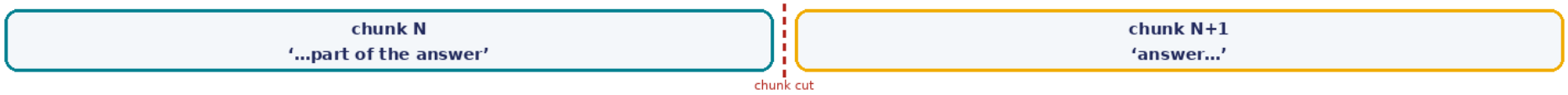
Example. A real run of the demo rig: a document about cybernetics, 587 characters.

Question: "What kinds of sources can information flows come from in Gordon Pask's definition?" The gold answer is "from any sources", positions 528–547: a literal substring of the document at that spot, not invented by anyone.

There is no model at this step. None of the four cutting versions calls an LLM or an embedding — all four "Chunking" nodes (next beat) are plain code.

The model appears later, at the step of embedding an already-made chunk (Case 1), which this measurement does not touch. There are no results on this frame — what came out of this is next.

What the run showed across five chunk sizes



What counts as "broken". The gold answer sits in the document's text at known positions; if a chunk cut happens to fall inside that span, the answer ends up in two different chunks, and neither carries it whole — the model gets only half.

72 questions, headline chunk size — 180 characters. Version 1 — 9 broken (12.5%), 63 intact (87.5%). Version 2 — 1 broken (1.4%), 71 intact (98.6%).

CHUNK SIZE, CHARACTERS	V1 — BY CHARACTER COUNT	V2 — BY SENTENCE BOUNDARIES	V3 — BY WORD BOUNDARIES	V4 — CHARACTER COUNT + OVERLAP
120	9.7% (7 of 72)	1.4% (1 of 72)	9.7% (7 of 72)	5.6% (4 of 72)
180 (headline)	12.5% (9 of 72)	1.4% (1 of 72)	8.3% (6 of 72)	5.6% (4 of 72)
220	4.2% (3 of 72)	1.4% (1 of 72)	1.4% (1 of 72)	1.4% (1 of 72)
300	6.9% (5 of 72)	0.0% (0 of 72)	5.6% (4 of 72)	1.4% (1 of 72)
450	2.8% (2 of 72)	1.4% (1 of 72)	0.0% (0 of 72)	0.0% (0 of 72)

What the measurement of the final answer showed

What we're measuring, and on what. The outcome of the entire pipeline: top-K search among the same 60 documents and 72 questions as in the previous beat, then `gpt-4o-mini` answers from the retrieved chunks — scored by the same criterion as Case 1. The previous beat measured something different: span integrity inside one chunk. Here, only the cutting rule differs; the search model and the answering model are the same everywhere.

K — how many chunks are handed to the model. K=1 — one, the most similar chunk. K=5 — five similar chunks at once.

At K=1, version 2 answers correctly in 52 of 72 cases (72.2%) against 41 of 72 (56.9%) for version 1 — fifteen and three tenths percentage points of gap, the same criterion as in Case 1.

CUTTING VERSION	K=1	K=2	K=3	K=5
v1 — by character count	56.9% (41/72)	59.7% (43/72)	63.9% (46/72)	73.6% (53/72)
v2 — by sentence boundaries	72.2% (52/72)	83.3% (60/72)	77.8% (56/72)	80.6% (58/72)
v3 — by word boundaries	55.6% (40/72)	58.3% (42/72)	63.9% (46/72)	63.9% (46/72)
v4 — character count + overlap	55.6% (40/72)	65.3% (47/72)	66.7% (48/72)	70.8% (51/72)

The gap narrows, but doesn't disappear

The gap narrows by more than half. At K=1, version 2 outruns version 1 by 15.3 percentage points (72.2% against 56.9%). At K=5 — by 7.0 percentage points (80.6% against 73.6%): more than twice as narrow.

Version 1 needs roughly five times more candidates. To get close to the result version 2 already delivers at K=1 (72.2%, 52 correct answers out of 72), version 1 needs to return five candidates instead of one: at K=5 it reaches 73.6% (53 of 72) — nearly the same number, but at the cost of a fivefold increase in K. Version 2 remains the better version at every one of the four tested K without exception.

Breakdown on a real document: where the cut fell

Document (the same example as in the test structure): "…Gordon Pask broadened the definition, including information flows from any sources, from stars to the brain." Gold answer — "from any sources", positions 528–547.

VERSION	WHERE THE CUT FALLS NEAR THE ANSWER	VERDICT
Cut by character count (180)	chunk 3 ends on "…from any sour", chunk 4 begins with "ces…" — the cut at character 540 fell inside a word	**BROKEN**
Cut by sentence boundaries	the whole answer sits inside one chunk [411–587): "…Gordon Pask broadened the definition, including information flows from any sources…"	**INTACT**
Cut by word boundaries	the chunk boundary sits exactly at character 528 — right at the start of the answer; chunk 4 [528–587) begins with the word "sources" and carries the whole answer	**INTACT**
Cut by character count with overlap	overlap shifts the start of the last chunk back to character 459; the whole answer (528–547) fits inside the chunk [459–587)	**INTACT**

Version 1's systematic error — and the lesson of one document

Version 1's systematic error. Cutting by character count doesn't look at the text at all — the chunk boundary can fall anywhere, including inside a single word.

Here, character 540 of the document happened to land exactly in the middle of the word "sources".

One document illustrates the mechanism; the conclusion for the whole sample comes from the summary. On this document, versions 3 and 4 keep the answer intact.

But the summary across sixty documents (previous beat) still shows them, on average, noticeably worse than version 2 — 8.3% and 5.6% broken answers against 1.4%.

Conclusion. This isn't a one-off failure on a single document — the summary across sixty documents and seventy-two questions shows the same effect on average.

Version 1 systematically tears answers in half; version 2 almost never does.

What the measurement confirmed for each of the four actions

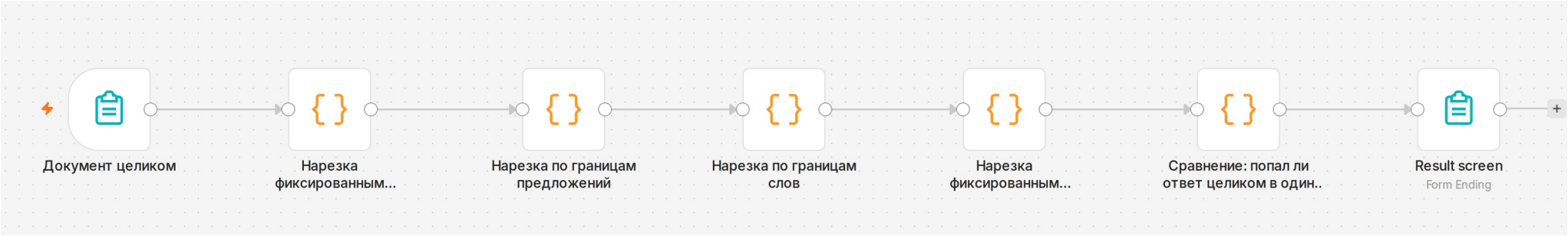
ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Cut by character count	The document is chopped into pieces of a set length with no regard for the text inside	Simplest to implement — one parameter, one pass	Breaks the answer to a question in 2.8–12.5% of cases depending on size; there's no way to tell in advance which size is "more convenient" without checking
Cut by sentence boundaries	The cut falls only where a sentence ends	The share of broken answers drops several times over — almost always 0–1.4% across all tested sizes	A bit more involved to implement (needs text parsing into sentences — character counting alone isn't enough); depends on the quality of sentence splitting for the specific language and text
Cut by word boundaries	The document is split into words, a chunk accumulates to the target size, the cut only falls at a word boundary	A cheap alternative to version 1: keeps the word whole and is simpler to implement than parsing sentences	Breaks the answer in 0–9.7% of cases: holds the word boundary, but knows nothing about sentence boundaries
Cut by character count with overlap	The same logic as version 1, but neighboring chunks overlap by part of the chunk's length	Breaks the answer less often than version 1 — 0–5.6% against 2.8–12.5%	The store grows from character duplication; doesn't close the gap to version 2's level

Where the chunk boundary should fall

A chunk's boundary must coincide with a boundary of meaning inside the document — character count has no bearing here. Cutting that slices the document "however it falls" saves one preparation step, and pays for it by having part of the answers to future questions end up torn between two chunks.

This decision doesn't need a model. All four cutting versions — by character count, by sentence boundaries, by word boundaries, and by character count with overlap — are purely algorithmic: the boundary's position is computed by code, with no call to an LLM.

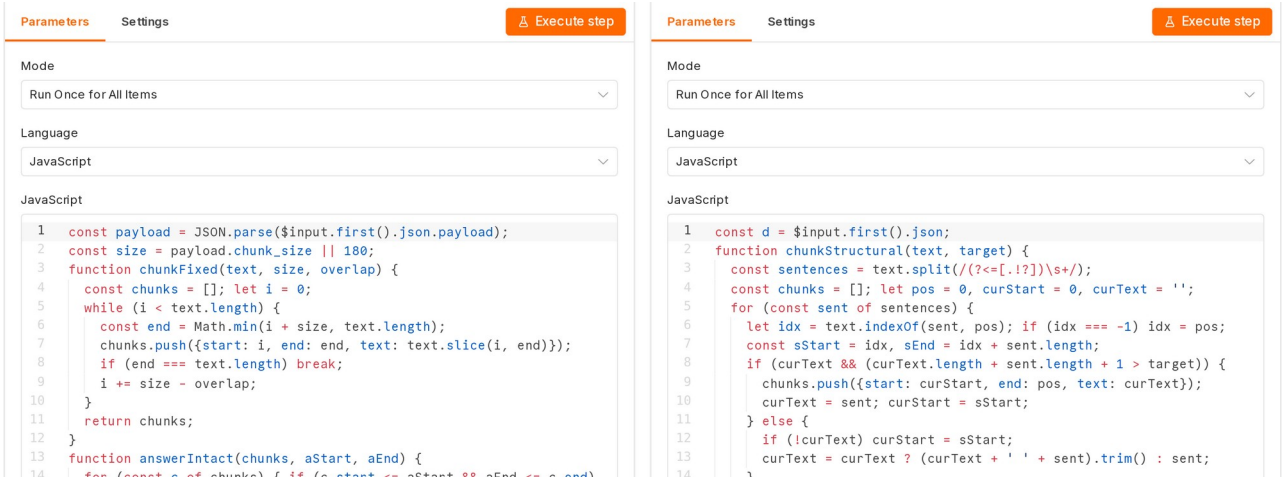
The real canvas: comparing four versions on one document



Whole document → Fixed-size chunking → Sentence-boundary chunking → Word-boundary chunking → Fixed-size chunking with overlap → Comparison: did the answer land whole in one chunk → result screen.

This canvas has no embedding step and no store write — it's a separate demo scenario that runs all four chunking versions on one document and compares them right on screen. The chunk-embedding step (the `text-embedding-3-small` model, Case 1) sits in a different scenario, after the chunk is already made, and doesn't change with any setting depending on which chunking version produced it.

The real screen: how the two chunking nodes are built



Fixed-size chunking (left) reads `payload.chunk_size` (default 180), then in a `while (i < text.length)` loop cuts the string into consecutive chunks of this many characters, with no regard for content — after each chunk it advances by `size` characters and repeats. Sentence-boundary chunking (right) first splits the text into sentences using a regular expression (a boundary after a period, exclamation mark, or question mark), then accumulates sentences into one chunk until the total length exceeds `target` characters — the chunk only breaks off at a sentence boundary, never inside one. Both nodes are plain code (a Code node, JavaScript language, "Run Once for All Items" mode), with no call to a model at all.

What Olga sees, and what the requester sees

The requester sees the result directly: if the answer to their question is torn between two chunks, the model gets only half of the needed fact and answers incompletely or incorrectly — this isn't a matter of search quality; what it found was itself incomplete. Olga sees a number — what share of questions risks getting a broken answer under the current cutting setting.

The principle is general: applicable to any text-cutting tool, and the chunking node in n8n is just one example here. A store chunk is a unit inside which meaning must be preserved; the choice of where to cut directly determines how many answers arrive intact, and it carries far more weight than an ordinary technical setting.

What a fuller measurement showed

Chunk size doesn't behave monotonically on its own. For version 1, the share of broken answers doesn't fall smoothly as chunk size grows: 9.7% → 12.5% → 4.2% → 6.9% → 2.8%. Choosing a "nice-looking" size doesn't guarantee quality — checking against your own data is required: a borrowed empirical rule doesn't carry over.

Overlap gives a partial gain. At the headline size, adding overlap cuts version 1's share of broken answers almost in half — from 12.5% to 5.6%. It doesn't close the gap to version 2's level (1.4%).

Word-boundary cutting shows the same picture. At the same size, the share of broken answers drops from 12.5% to 8.3% — the word is protected from being torn, the sentence still isn't.

The session's second axis line is closed: a document CAN technically be cut into chunks any way you like — but what share of future answers arrives intact depends directly on this, and cheap fixes for the symptom reduce that share without removing it.

Case 3. Choosing a Search Method

Case 2 split documents correctly, by meaning — counting characters played no part in it. But a correctly split chunk still has to be found among hundreds of identical-looking ones once a specific question arrives.

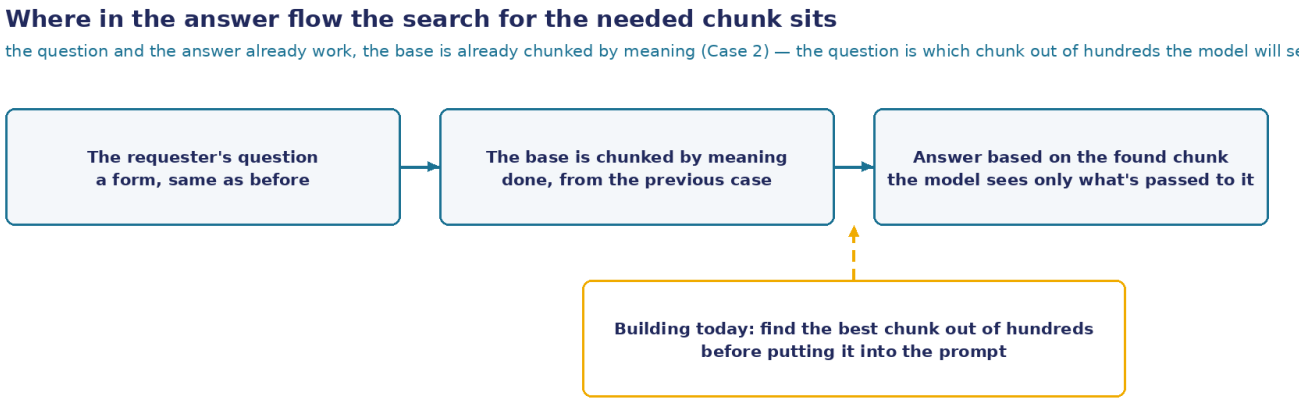
Хранение базы знаний

Разбиение документа на части

Чем искать нужный кусок

Поиск по изображениям

How to find the right chunk among hundreds of identical ones



The need. Among hundreds of correctly split chunks of the base, find exactly the one holding the answer to the given question — and find it right in first place in the results — tenth place doesn't count here.

Version 1 — after what Cases 1 and 2 showed (semantic search works, splitting by meaning works), it's natural to assume: search should also go by meaning — through vector comparison of the query against the base's chunks. It seems that since such a search understands the meaning of the question, it must surely be better than keyword search.

This is exactly the intuition this case checks by measurement.

What to search the needed chunk with, among hundreds

What should search the needed chunk among hundreds?

- **full-text search (by words)**
- **vector search (by meaning)**
- **hybrid search (words + vector)**
- **reranker (model re-ranking)**

What each of the four actions does

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Full-text search (keyword, BM25)	Matches tokens of the query and the document, no model — the weighting function is called BM25	Cheap and fast — the search itself costs nothing	Finds word matches — it doesn't recognize meaning: a lexical gap between the question and the document, as well as documents similar in topic but different in substance, throw off the exact match
Vector search (by embeddings)	The query's embedding is compared against the embeddings of the base's chunks by cosine similarity — the nearest one wins	Finds by meaning: isn't limited to word overlap — rephrasing the question in different words doesn't get in the way	Weaker where exact matching of a specific name or number matters — closeness of embeddings doesn't guarantee an exact match of a detail
Hybrid search (RRF)	Two lists — by words and by embedding — merge into one via the Reciprocal Rank Fusion (RRF) formula	Combines the strength of both signals where they diverge — takes whatever either of the two methods found	A weak word signal sometimes pulls the merged result toward where a pure vector would not have gone on its own

How RRF computes a single list: an example on three candidates

The principle in words. Each document in both lists — by words and by embedding — earns a score: the higher the rank, the larger the score. The order is built on the sum of a document's scores from both lists at once.

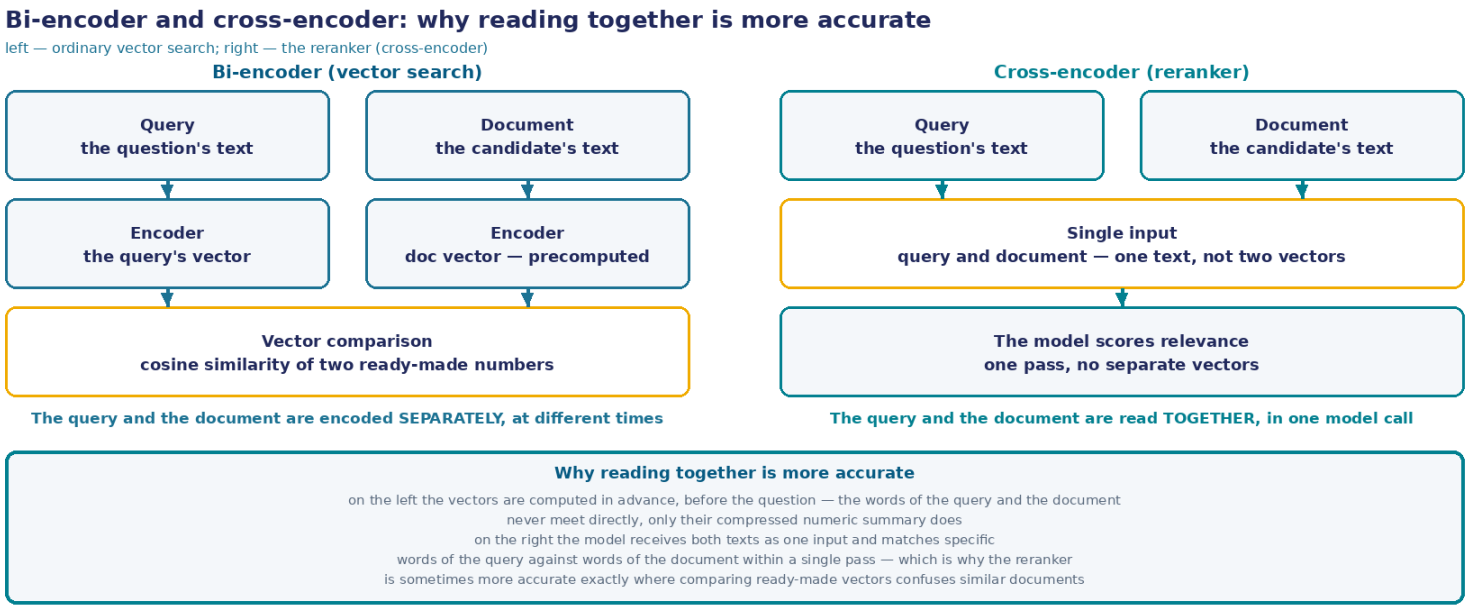
CANDIDATE	RANK BY WORDS	RANK BY MEANING	OUTCOME AFTER RRF
Candidate A	2nd	4th	2nd
Candidate B	3rd	2nd	1st
Candidate V	4th	3rd	3rd

Candidate B was never first, either by words or by meaning — but it was never weak anywhere either, and that was enough to beat both rivals, each of which had one strong rank and one weak one.

What the fourth action does: a model-based review

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Reranker (model-based review)	Once a list of candidates has already been assembled by one or more search methods, a separate model call reads both the question and all the candidates in full, and reviews their order	The model sees the candidates in full and can tell the right document apart from similar neighbors where statistical comparison of words or meaning gets confused	A separate model call is more expensive than any of the steps without a model; it only makes sense once a list of candidates already exists — so it's a step on top of an already-assembled result, not something you can call a standalone way of searching

How the reranker works inside: bi-encoder vs. cross-encoder



A bi-encoder encodes the query and the document separately and compares already-finished vectors. A cross-encoder (reranker) feeds them into the model together, as a single text, and reads the direct interaction of the words — exactly what separate encoding compresses into a number before the question even existed.

Test structure: five hundred documents, fifty questions, three groups

Corpus. 500 SberQuAD documents, including 8 confusable clusters — groups of documents that split one and the same rare entity. Example: the "Zvyozdny" cluster — one document about spacecraft, another about concert programs from the mid-1980s; both use the word "zvyozdny" ("starry"), but answer completely different questions.

GROUP	HOW MANY	EXAMPLE QUESTION	EXPECTED ANSWER	WHY THIS GROUP
Lexical gap	16	"When did the battle take place?"	April 16, 1746	The question and the document share almost no words — the expected zone of semantic search
Exact match	16	"Who took the title of Duke of Milan in 1395?"	Gian Galeazzo Visconti	The year and the title from the question sit literally in the text — the expected zone of keyword search

Test structure: the third group and the search model

GROUP	HOW MANY	EXAMPLE QUESTION	EXPECTED ANSWER	WHY THIS GROUP
Neutral	18	"What was barter exchange in trade?"	The only available form of trade	The class is picked out by the absence of a shared number with the text — the degree of lexical similarity doesn't matter here. The group is neutral: neither a gap nor an exact match, although it also contains questions lexically close to the text

Semantic search is computed by `text-embedding-3-small` via OpenRouter — the same embedding model that already worked in Case 1 of this session, cosine similarity against all 500 documents, with no cutoff threshold. The metric is the same for every variant — recall@1 and recall@3 (whether the correct document was found first, or among the first three) and the average rank of the correct document, against the SberQuAD gold labels. There are no results on this frame — next, we look at what came out.

What the run on 500 documents and 50 questions showed

Best result for first place: semantic search finds the correct document right in first place in 43 of 50 questions (86%).

METHOD	RECALL@1	RECALL@3	AVERAGE RANK OF CORRECT (OUT OF 500)
Search by words	0.74	0.80	9.9
Search by meaning (vector)	0.86	0.92	2.0
Search both, merge (hybrid, RRF)	0.84	0.94	1.6

Recall@1 by the three question groups

A summary across the same three groups the previous beat named (recall@1):

QUESTION GROUP	WORDS	MEANING	HYBRID
Lexical gap (16)	0.25	**0.94**	0.69
Exact match (16)	**1.00**	0.81	0.94
Neutral (18)	**0.94**	0.83	0.89

A systematic finding: the hybrid does NOT win in a SINGLE one of the three groups on its own. Its aggregate result (0.84) comes not from being the best anywhere, but from nowhere being the worst.

What the measurement confirmed for each of the three signals

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Search by words	Matches tokens of the query and the document, no model	Cheap and fast — the search itself costs nothing	recall@1 = 0.74 — the worst of the three; if it's not in first place, it sits on average at the 9.9th position out of 500 — the lexical gap and the confusable clusters throw off the exact match
Search by meaning (vector)	The query's embedding is compared against the embeddings of the base's chunks	recall@1 = 0.86 — the best result among methods with no model at the ranking step; average rank 2.0	recall@3 (0.92) is a touch below the hybrid's; weaker where exact matching of a name or number matters
Search both, merge (hybrid, RRF)	Two independent rankings merge into one via the Reciprocal Rank Fusion rule	Best recall@3 (0.94) and average rank (1.6) among methods with no model at the ranking step	recall@1 = 0.84 — lower than pure vector's (0.86): a weak word signal sometimes pulls the merged vote away from where vector was right straight away

The next step after the hybrid: review by the model

Both bases are computed by `text-embedding-3-small` (search by meaning) — the reranker on top adds a separate `gpt-4o-mini` call.

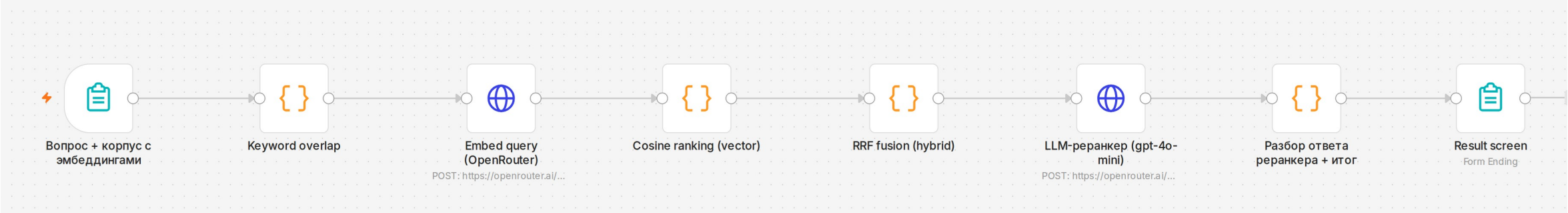
BASE + RERANKER	RECALL@1	$/QUESTION (SEARCH)
Hybrid + LLM reranker	**0.96** (was 0.84)	$0.000329
Vector + LLM reranker	**0.96** (was 0.86)	$0.000327

Example of a fix. The question "What were the most popular works created in 1984–1986" (gold document — a member of a confusable cluster): pure vector found it in 10th place, the hybrid in 2nd place; the reranker raised both variants to 1st place.

Search is a merge of at least two signals

Search is a merge of at least two different relevance signals: one signal alone isn't enough, before deciding which chunk to put in the prompt. One signal (words) has its own zone where it wins — exact matches, names, numbers; the other (meaning) has its own — retelling in one's own words. After merging the signals comes an optional extra step: hand the final order off for review by the model, if the price of the separate call and the added latency are justified by the cost of a ranking error.

The real canvas: from question to reranker



Keyword overlap computes the ranking by words as the first node. Next come the query's embedding and cosine ranking by meaning. Only once both lists are ready does RRF fusion merge them — and only then, as an optional step, does the LLM reranker enter the picture.

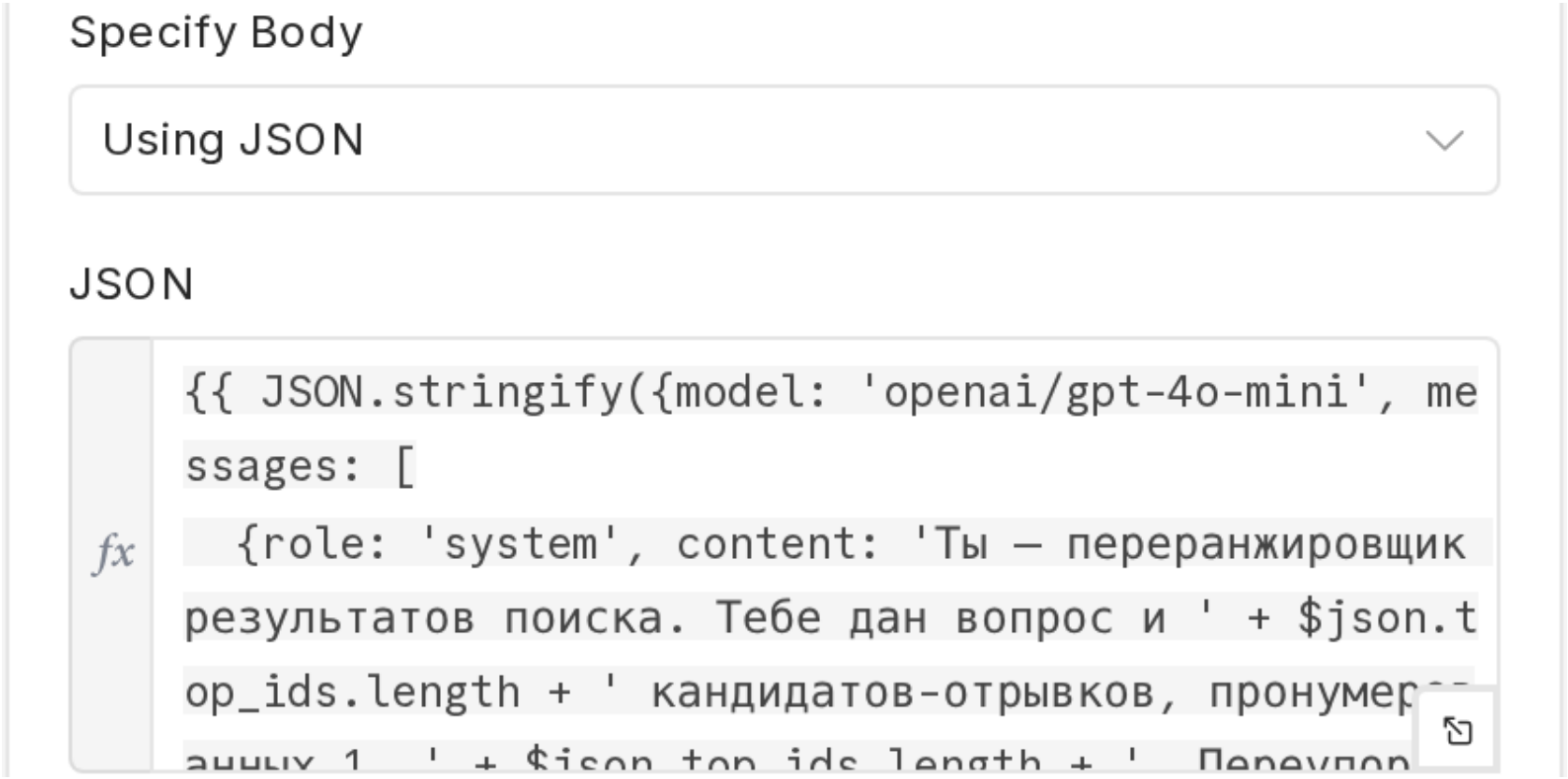
The real panel: the merge node's code

JavaScript

```
1  const d = $input.first().json;
2  function rrf(rankA, rankB, k=60) {
3    const scores = {};
4    [rankA, rankB].forEach(rank => { rank.forEach((id, i) => {
5      scores[id] = (scores[id]||0) + 1/(k+i+1); }); });
6    return Object.keys(scores).sort((a,b)=>scores[b]-scores[a]);
7  }
8  const hybrid_rank = rrf(d.keyword_rank, d.vector_rank, 60);
9  const TOP_N = Math.min(10, hybrid_rank.length);
10 const topIds = hybrid_rank.slice(0, TOP_N);
11 const byId = {}; d.corpus.forEach(c=>byId[c.id]=c.text);
12 const candidatesText = topIds.map((id,i)=>`${i+1}.
   ${byId[id]}`).join('\n\n');
13 return [{json: {...d, hybrid_rank, top_ids: topIds, candidates_text:
   candidatesText}}];
```

Line 8 is the whole idea of the node: `rrf(d.keyword_rank, d.vector_rank, 60)`. Two already-ready lists come together into one by the rule "the higher a document stands in both lists, the higher it stands in the merged one."

The real panel: the reranker's request body



> *"You are a search-result reranker. You are given a question and 10 candidate excerpts, numbered 1..10. Reorder the candidate numbers from most relevant to the question to least relevant. Reply with STRICTLY a JSON array of numbers..."*

What Olga sees, and what the requester sees

To the requester, hybrid search looks like a more stable answer regardless of exactly how the question is phrased — with an exact term or in their own retelling. To Olga, a number is visible: the share of questions for which the process found the correct document among the top results is higher for the hybrid than for either of the two methods on its own.

The principle transfers to any orchestrator: semantic search doesn't replace keyword search, and neither does the reverse — each has its own class of questions where it's stronger, and merging both results doesn't require knowing ahead of time which class of question will come next.

When the hybrid isn't needed, and when the reranker fails

Criterion. Hybrid search is a candidate for cutting within Case 3 (item №2 of the general cutting order, after cutting the detailed perceptual-hash breakdown in Case 4) if the slot doesn't fit all the material: the method works correctly, but on a small, uniform base it isn't needed to the degree it's needed on a large, heterogeneous one.

WHEN IT'S NEEDED

The base is large, and questions are phrased both with exact terms and in retellings in one's own words

The cost of a ranking error is high — the requester gets a wrong answer from the second- or third-ranked candidate

WHEN IT ISN'T

The base is small and uniform in how its questions are phrased

The difference between a single method and the merge of two is statistically indistinguishable on this base

The reranker's boundary: the gain isn't on every question

The reranker is a second boundary of the same kind, with two sides at once. It adds 10–12 percentage points to recall@1 — but it costs a separate model call ($0.00033, more expensive than the entire Case 1 pipeline) and latency (+1.6–1.9s). It can raise the gold document from 10th place to 1st — that's what happened with the question about popular works from the mid-1980s. And it can just as well knock an already-correct result from 1st place to 3rd — that's what happened with the question "An example of water barriers?", where pure vector had already found the correct document first. If the document isn't even among the top-10 candidates before the rerank — as in the question about barter exchange in trade, where vector found the gold document only in 26th place — the reranker won't raise it at all: it only rearranges what it was shown, it doesn't touch the whole pool of documents again.

The third axis line of the session is closed: semantic search finds the document in first place more often than keyword search on a tough corpus — but not guaranteed and not universally; it depends on the corpus and the questions, and merging signals doesn't win on every metric at once.

Case 4. Image Search

Two sessions ago showed the limits of handing a model a picture. Last session — that the model can read it. Today — the same storage-and-search task as in the three previous cases, just for scans and photos.

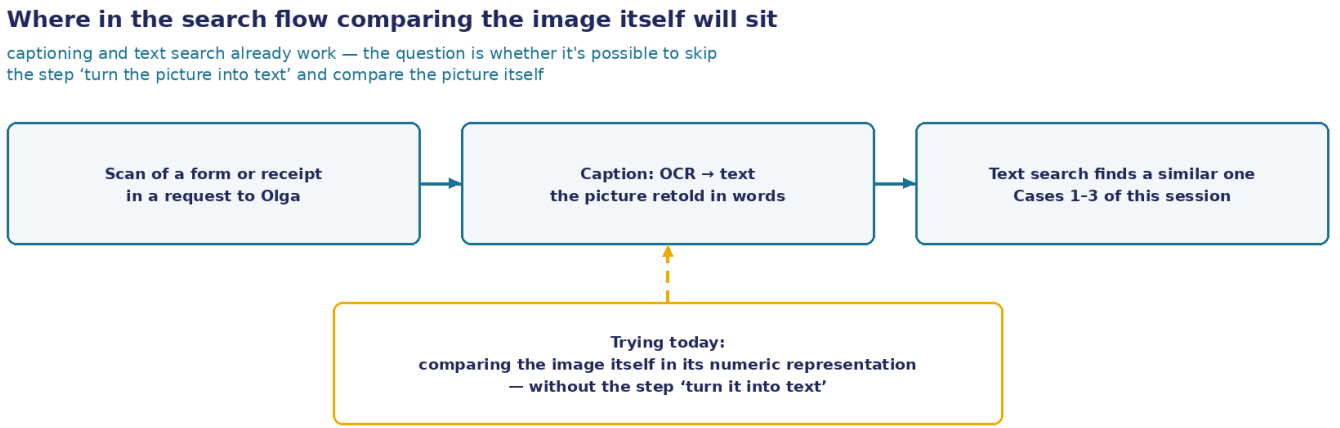
Хранение базы знаний

Разбиение документа на части

Чем искать нужный кусок

Поиск по изображениям

How to find a similar past scan



The need. The training center gets scans of forms and receipts in requests regularly. We need to find a similar scan among past requests: reading the current one is not enough here.

Version 1 — the natural first attempt, a direct continuation of the three previous cases: caption the picture in text (recognize the text, or get a text description from a model) and search that caption with the same mechanism as in Case 3 — as an ordinary text document.

One of the three options tested today will end in an honestly shown failure — a real failure with a learned lesson; calling it a weakened version of success would be wrong.

How to search for a similar scan among the past ones

How do you search for a similar scan among the past ones?

caption the picture in words and search as text

search the picture by the picture directly

compare pictures by a short fingerprint, without calling a model

What each of the three actions does

ACTION	HOW IT WORKS	ADVANTAGE	DRAWBACKS
Caption and search as text	The receipt's text turns into the same text embedding as in Cases 1-3	Works correctly: embeddings differ meaningfully between receipts, reuses the entire mechanism of Cases 1-3	A caption is a word retelling of the picture: field layout, the visual look of the form, the stamp, a handwritten note are lost before they ever reach search
Search picture by picture (multimodal embedding)	The model turns the picture itself into a numeric representation, with no text in between	In theory loses nothing of what a caption discards	**Doesn't work on this rig at the scan's real resolution — breakdown next beat**
Image fingerprint (perceptual hash)	The picture is compressed into a short bit string without calling a model; closeness is matching bits between fingerprints (Hamming distance)	Reliably catches near-duplicates under recompression — without calling a model, fast and local	The fingerprint stores only outward appearance: different documents occasionally give a close fingerprint by chance, meaning-level duplicates with a different layout — different. Good for "the same file again"; doesn't solve "a similar document"

Failure, part 1: the plan's model doesn't exist

Why this beat exists. A provider's model list can't be relied on in advance when planning: the catalog changes without notice. The only reliable check is a direct API call by a specific model id; this breakdown of the failure starts with exactly that.

Step 1. `voyage-multimodal-3`, the model the plan originally counted on, wasn't found on this rig's OpenRouter under any of the checked spellings of the id. The full model catalog (452 entries) contains not a single model with "voyage" or "embed" in the name at all.

A search of real candidates found exactly one working multimodal model — `amazon/titan-embed-image-v1` — it accepts both text and a picture, and returns a vector without an error.

Structure of the measurement: one pair of receipts, three resolutions

The call. `POST https://openrouter.ai/api/v1/embeddings`, `model: amazon/titan-embed-image-v1`, input — a single `data` URL string (`…`), with no separate text field.

RESOLUTION	JPEG QUALITY	APPROXIMATE BASE64 SIZE
300×450 (real scan)	q80	~18,000–26,000 characters
128×192 (preview)	q75	~4,300–4,600 characters
64×96 (icon preview)	q80	~6,000–8,000 characters

Structure of the measurement: what result is expected

The expectation, if the model really sees the picture: a plain error-free response isn't enough here. Two DIFFERENT receipts at the same resolution must give two DIFFERENT vectors — the cosine distance between them must differ from 1.0. A check of "did the response come back in the right shape" doesn't show this: it passes the same way whether the model honestly uses the picture or ignores it entirely.

Summary and breakdown: no resolution gave reliable distinguishing

Summary. At the resolution actually needed by the task — failure. The model doesn't use the picture at either of the two tested real-scan resolutions (300×450, 128×192) — a difference only shows up at the smallest tested size, and even there it's statistically indistinguishable from noise.

RESOLUTION	VERDICT	WHAT THE TEST SHOWED
300×450 (real scan)	**FAILURE — IDENTICAL VECTOR**	cosine = 1.0 to 9 decimal places for any two different receipts
128×192	**FAILURE — IDENTICAL VECTOR**	cosine = 1.0 — confirmed by a live repeat check (2026-11-02)
64×96 (small preview)	**FORMALLY DISTINGUISHES, BUT ≈ NOISE**	correlation of visual similarity with textual similarity across all pairs: −0.077 — statistically indistinguishable from zero

An error-free response doesn't prove the input was used

An error-free response and a processed input are different things. The only way to check this is to feed in two knowingly different pictures and compare whether the output differs; a test of "did the response come back in the right shape" won't show it.

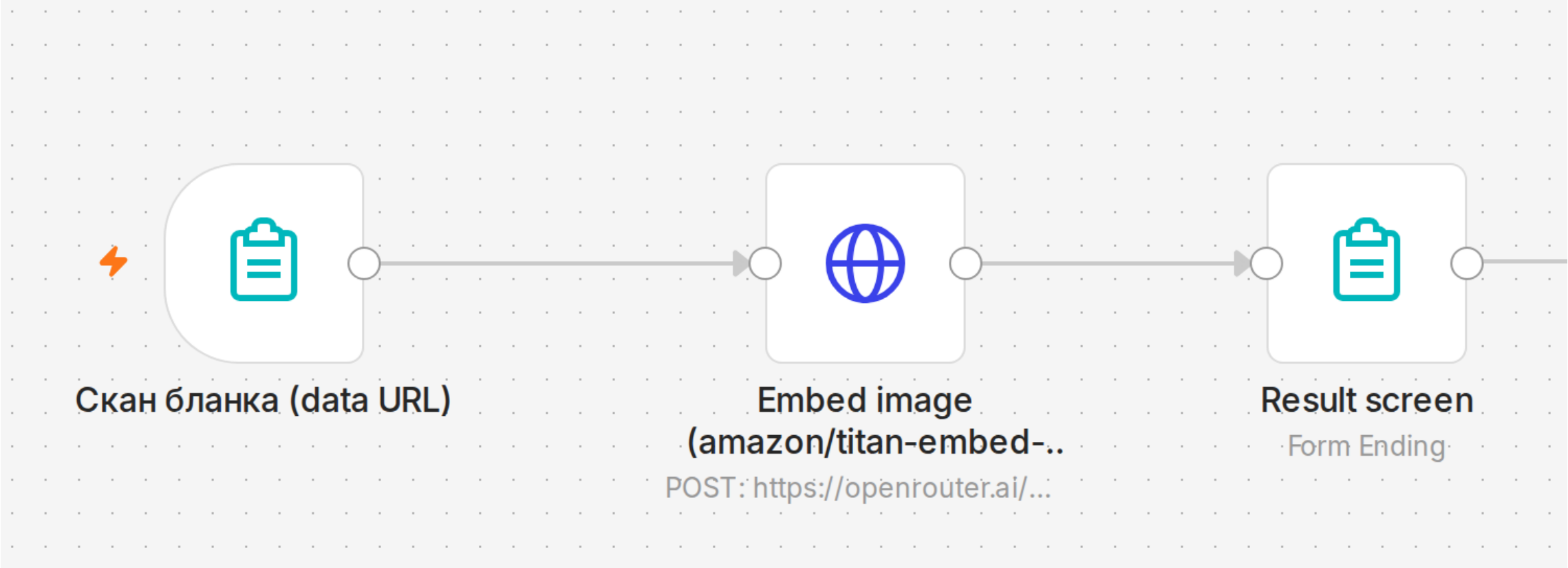
For comparison, in a different unit. Perceptual hash (the classic, model-free technique) on these same receipts confidently tells a duplicate apart from a random stranger receipt: 0–1 bit Hamming distance to the duplicate against 14–49 bits to a random stranger receipt. This isn't a row in that table: Hamming bits and cosine are different units (a detailed breakdown of phash's own boundary — in this case's boundary beat, further on).

A correct idea without a ready-made realization

The correct architecture is to compare pictures in their own numeric representation: a text intermediary isn't enough here. This is an architectural thesis, and it stays true regardless of which specific model implements it.

The working solution to today's rig's task is assembled from the two remaining parts of the breakdown: caption and search as text — for semantic search, "a similar document by content" — and perceptual hash as a separate, fast step — for the task "literally the same scan, just recompressed." This is the same rule already met in Case 3; calling it a compromise instead of multimodal embedding would be wrong: one signal doesn't cover every case.

The real canvas: an attempt that didn't succeed



The canvas on the rig is really assembled and really executed — but layer 1's architectural thesis isn't confirmed by this run: there's no working demonstration of "search picture by picture" on this rig, only an honestly shown and reproduced failure.

The real panel: the request body is an expression, not text



`Body Content Type: JSON` → `Specify Body: Using JSON` → `JSON: {{ $json.body }}` — the same request-body setup the text-embedding node already showed in the first case of this session, only here `$json.body` holds a picture, with no text.

What Olga sees, and what the requester sees

The requester doesn't see exactly how the process found a similar past form — they see that it found it, when that's possible by content, and that the system separately and reliably tells a literal repeat scan of the same request apart from a new document. Olga sees the boundary: for one class of similarity (semantic — "the same type of form") one mechanism is needed, and for another (literal — "the same document, just re-shot") a different one is needed, and it's cheaper.

A principle, portable to any orchestrator: the architectural idea "compare data in its own space, bypassing a retelling" can be correct and still have no ready-made working realization with a specific provider at a specific moment — checking this takes one controlled experiment with two different inputs, and skipping that experiment means carrying out of the session an unworkable solution that looked workable on the first test.

Where the exact-repeats method stops being reliable

RECEIPT	SOFT DUPLICATE (BITS OUT OF 64)	HARD DUPLICATE	RANDOM STRANGER
receipt_07	1	27	14
receipt_18	0	10	24
receipt_17	0	30	35
receipt_04	1	40	23
receipt_11	0	10	44
receipt_19	0	10	49

Boundary of applicability. Under rotation, the distance grows to 10–40 bits — comparable to a random stranger receipt in half the receipts. The method solves the subclass "same scan, recompressed"; rotation doesn't fall into that subclass.

When to try searching picture by picture directly

When it's worth trying search picture by picture directly — and when it's definitely not, on this rig. Worth trying: when the provider and model confirm accepting the picture by a separate controlled test on two knowingly different inputs, at the exact resolution the task needs; one quick "did a response come back" check isn't enough here. Not worth it right now, on this rig: `amazon/titan-embed-image-v1` through OpenRouter at the real scan's resolution (300×450) — a confirmed failure from a controlled test; calling it an assumption would be wrong, and it's not a reason to avoid multimodal embeddings altogether either.

The session's fourth axis row is closed: if we know how to search text, we won't find a picture the same way: a caption loses what's visible, direct comparison of pictures can answer without an error and still not use the input at all.

The repeat test found a working model

Found by a discipline of selection. OpenRouter's catalog gave 9 candidates accepting a picture. 3 of 5 formally told two random images apart, but failed the margin test — a recompressed duplicate and a stranger document came out at almost the same distance for them. `nvidia/nemotron-3.5-embed-1b:free` passed both checks reliably.

METHOD (ALL 30 CORD SCANS)	SOFT DUPLICATE, RECALL@1	HARD DUPLICATE (ROTATION), RECALL@1
Working model (nemotron-3.5-embed-1b)	100% (30/30)	**96.7% (29/30)**
Perceptual hash	96.7% (29/30)	**23.3% (7/30)**

n72's boundary got a numerical answer. Under rotation, across all 30 scans — 96.7% correct hits for the working model against 23.3% for the fingerprint.

There is a signal on semantic similarity, but it's limited

METHOD	AGREEMENT WITH THE TEXT REFERENCE, RECALL@1	CHANCE LEVEL	CORRELATION WITH TEXT
Working model (nemotron-3.5-embed-1b)	**26.7% (8/30)**	3.45% (1/29)	0.529
Perceptual hash	3.3% (1/30)	3.45% (1/29)	−0.071

Honest conclusion. The working model carries a measurable signal about the picture's meaning — recall@1 almost eight times above chance level, and a noticeable positive correlation with text similarity (0.529 against −0.077 for the broken titan). Text search by caption itself, which serves as the reference here, finds the semantically similar receipt in 96.7% of cases — direct comparison of pictures doesn't reach that bar. Perceptual hash is insensitive to meaning altogether: its result is at chance level, 0.033 against 0.0345 — exactly what n66/n71's boundary predicted.

The one experiment: a model with no OpenRouter at all

A separate question from the session's plan. A provider's model list changes without warning — the same problem that opened this section at n61. CLIP is the local industry standard for picture embedding, with no provider catalog at all. The one such experiment in this section.

METRIC, RECALL@1	CLIP, LOCALLY	API (CONTEXT)	PHASH (CONTEXT)
hard duplicate (rotation)	**76.7% (23/30)**	96.7% (29/30)	23.3% (7/30)
semantic similarity	**13.3% (4/30)**	26.7% (8/30)	3.3% (1/30)

Honest conclusion. CLIP works locally, in seconds, with no network and no provider catalog — and on this measurement is noticeably weaker than the API candidate on both metrics. Independence and speed are its benefit; it doesn't cover accuracy.

Relevant ≠ Correct

Today's four cases all taught finding the needed knowledge-base chunk and putting it in the prompt before answering. The whole time, standing just offscreen, was an assumption never explicitly checked: that if the system found a chunk topically related to the question, the answer built on it would turn out correct.

Incident 1. Google AI Overviews and the Pizza Advice

In 2024, Google Search's AI-generated-overviews feature several times served users advice drawn from sources that sounded credible in form but were, in content, a joke: one widely discussed incident was a recommendation to add non-toxic glue to pizza sauce, whose source turned out to be an old joking comment on an internet forum.

The system found text that really was about exactly that — about cheese, about pizza, about how to make it less runny. By search standards — a success: high relevance. By the standard of the answer the user got — a failure: the source turned out to be a joke, not a cooking recommendation.

Incident 2. New York City's Municipal Chatbot

A separate incident from 2024, documented by journalistic investigation: New York City's municipal chatbot, built to help small-business owners, several times gave direct questions about city rules confident, grammatically flawless answers that were legally incorrect or directly contradicted city law — for example, the bot said employers could keep employees' tips for themselves, and that landlords could refuse tenants with housing subsidies.

Both incidents technically worked correctly: they found text relevant to the topic and built a grammatically correct answer on it. The failure sits in the assumption that topical closeness equals factual correctness — the search mechanics have nothing to do with it.

Why This Concerns All Four of Today's Cases

Neither Case 1 (storage), nor Case 2 (chunking), nor Case 3 (search), nor Case 4 (image search) is automatically insured against this just by the fact that they use search instead of handing the model the entire volume of data at once. A search that correctly found the chunk closest in meaning — exactly what the recall and rank metrics measured across all four cases — only answers the question "is this similar to what was asked"; it doesn't touch the question "is this correct."

This isn't a reason to give up on searching the knowledge base — giving it up would send the session back to prompt-stuffing, which Case 1 already showed to be a losing option as the base grows. This is a reason for a separate step that the next sessions on this arc build separately: quality evaluation and a kill switch for an answer built on material that was found.

Four Rows of the Axis: What They Turn Out to Be

LOOKS RELIABLE	WHAT IT TURNS OUT TO BE
if the base has grown — just list it all in the prompt, like on S9	Accuracy holds at one level (~0.7 at 10/50/200) with no warning sign, while cost grows ×17.0 over the same range — and at 500 documents the request is physically rejected (HTTP 400, context exceeded); keyword search holds recall@1=0.8 at every size, vector search holds 1.0 at 10/50/200 and 0.9 at 500, at ~$0.00005 regardless of size
you can cut a document into pieces any way you like, as long as the chunk ends up in storage	Fixed-size chunking tears the answer apart in 2.8–12.5% of cases; sentence-boundary chunking stays almost always within 0–1.4%; a chunk size that merely "looks convenient" guarantees no quality without checking it against your own data
semantic search (vector) finds the same thing as keyword search anyway, just smarter	On a hard corpus, vector search beats keyword search on recall@1 (0.86 vs. 0.74) — but combining the two signals doesn't win on every metric at once; a reranker raises recall@1 to 0.96 at the cost of a separate model call and its own mistakes
if we know how to search text, we'll find an image the same way, just by captioning it in words	A caption loses what's visible in the image; direct image comparison technically doesn't work on this rig — the model answers without an error, but returns the same vector for different images

Where does the process get its facts, and why does it consider them true?

The Session's Tally: What Was Measured, What Remains Unverified

Four cases, four independent measurements on real data — SberQuAD for text (Cases 1–3), CORD for images (Case 4). Not one of them on a made-up number. One of the four runs — multimodal embedding in Case 4 — ended in an honestly shown result that didn't work, and it wasn't passed off as a patched-up substitute.

Storage (Case 1) saves on the cost of each answer, but adds a data-preparation step and the risk of a stale index — the base's copy doesn't update itself when the source changes. Sentence-boundary chunking (Case 2) is more accurate than fixed-size chunking by character count, but is a bit more complex to implement and depends on the quality of sentence splitting. Hybrid search (Case 3) is more reliable than any single method, but costs an extra computation-and-merging step on every query — and isn't justified on a small, uniform base; a reranker adds yet another paid step on top. Image search (Case 4), where it's actually needed, requires a separately verified component for each class of similarity — and not one convenient, universal answer turned out ready to use without its own verification.

What's Next

Can the process not just answer, but act on the company's behalf?

The question the previous seminar opened with ("can the process answer the requester on its own") and the question today's opened with ("where does the process get its facts") together lead to the next step on the course's stepped arc: what happens when the process is given not just knowledge, but authority — the right not just to propose an answer, but to carry out an action on the company's behalf.

And what today's session doesn't fix: this day's interlude — "found" ≠ "correct" — isn't closed off either by taking the base outside or by the right to act. That's a separate rung: quality evaluation and a kill switch, further along the arc.